Implementing a VR-Enabled Bicycle Simulator: Integrating Physical Sensor and Actuator Data for a Realistic Simulation Experience

David Flaig(✉), Hannes Imschweiler, Jana Heimel, and Gerrit Meixner

UniTyLab, Heilbronn University, Max-Planck-Str. 39, 74081 Heilbronn, Germany
{david.flaig,gerrit.meixner}@hs-heilbronn.de,
himschwe@stud.hs-heilbronn.de

Abstract. This research deals with the implementation process of a virtual reality (VR) bicycle simulator that leverages state-of-the-art sensors and actuators to simulate the physical experience of cycling, providing realistic haptic feedback to enhance immersion. The simulator's novelty lies in its extensive integration of sensors and actuators, coupled with a robust data transmission architecture that facilitates real-time interaction between the VR simulation and physical hardware. This real-time pipeline enables precise reading and writing processes that ensure seamless feedback and control. Designed for usability, scalability, and maintainability, the simulator's modular architecture supports straightforward "plug-and-play" operation, making it accessible for diverse research applications focused on cycling behavior and safety in urban traffic environments. The bicycle simulator establishes a solid foundation for future studies, enabling the evaluation of general usability, realism, and immersive experience, which will contribute to advancing VR applications in cycling simulation research.

Keywords: Virtual Reality · Realistic Simulation · Bicycle

1 Introduction

Over the past several decades, there has been a discernible surge in the proportion of bicycle traffic, particularly within major urban centers across Europe and America [13]. This trend has been further amplified by the onset of the COVID-19 pandemic, which catalyzed a notable "cycling boom" as individuals sought alternatives to traditional modes of transportation to mitigate the risk of contracting the virus, notably by avoiding crowded public transportation systems [16]. The convergence of environmental consciousness, health awareness, and pandemic-related concerns has prompted a growing number of people to embrace cycling as a viable means of urban mobility.

However, in a study conducted in Australia, 56% of surveyed individuals, both men and women, reported apprehensions about sharing the road with

D. Singh et al. (Eds.): IHCI 2024, LNCS 15558, pp. 109–120, 2025.
https://doi.org/10.1007/978-3-031-88881-6_9

motorized vehicles. Similarly, 54% expressed concerns about the potential for injury in the event of a collision with a motorized vehicle. Another notable aspect was that 53% of respondents expressed concerns about aggressive behavior exhibited by drivers of motorized vehicles [12]. These apprehensions contribute to an increasing sense of insecurity among cyclists regarding road participation. Furthermore, a study conducted by the German Federal Statistical Office in 2022 revealed that such concerns may not be unfounded. It was found that the percentage of urban traffic fatalities involving cyclists rose to 31.3% in 2022. This is an increase of 4.5% compared to 2021 [1,14]. Thus, to encourage more people to engage in cycling and alleviate their fears, it is crucial to comprehend the uncertainties and apprehensions of cyclists. The research in this paper deals with the development and implementation of a realistic VR-based bicycle simulator, utilizing sensors and actuators to facilitate an immersive traffic simulation. After the research outlined in this paper, the bicycle simulator is anticipated to serve as a valuable tool for empirical studies regarding road safety and the analysis of test subjects' driving behaviors.

2 Related Work

The development of bicycle simulators has experienced significant advancements, offering cyclists and enthusiasts a highly realistic riding experience. Particularly in scenarios where dangerous situations are inherent, simulation offers a safer alternative for conducting research. As a result, simulators are now being utilized across a spectrum of settings, from home entertainment to professional training and research simulations. Advancements in this field have outgrown the use of static bicycle frames, embracing cutting-edge technologies such as Virtual Reality (VR) and advanced sensory for gathering data and improving realism. An early example of a professional implementation of a bicycle simulator is KAIST interactive bicycle simulator from 2001. The simulator consists of a bicycle mounted on a Stewart platform, which provides six degrees of freedom (DOF) of motion to the bicycle, along with a magnetorheological handle, a pedal resistance system to generate motion feelings, a real-time visual simulator and projection system, sub-controllers and an integrating control network [10]. This combination provides an environment where "the rider on the bicycle feels the motion and has the visual experience as if one is riding in the campus of the Korea Advanced Institute [...]" [10]. However, in 2001, visual fidelity-achieved through Vega at the time-posed a significant limitation. Since then, there have been substantial improvements in visual quality, even in less professional setups. More recent work that aligns with the objectives of this research project can be found in the Bachelor thesis by Hampus Norén from Malmö University [11]. The thesis describes a comparable approach, with a primary focus on traffic safety research. The simulation environment is constructed using the Unity game engine, creating a virtual road environment. The HTC Vive Pro 2, a high-definition head-mounted display (HMD), is used to immerse participants in the simulated setting. An interesting aspect of Noren's setup is the use of

a bicycle frame combined with a Wahoo KICKR Snap, an indoor-bike trainer, which is integrated to simulate the physical resistance encountered while pedaling, providing a realistic biking experience. Furthermore, the incorporation of a Vive Controller to track the steering angle of the bicycle introduces a low-cost method for capturing the bike's steering angle accurately. The capabilities for data analytics and performance tracking in Noren's setup are particularly interesting. These features are essential for evaluating participant performance and analyzing traffic safety behaviors in a controlled, simulated environment. In 2021, Ayah Hamad [9] introduced a simulator that utilizes a motion platform. The setup is mounted on a motion platform that enables side-to-side and fore-aft tilting movements. This part not only helps in reducing simulator sickness, but also significantly improves the simulator's realism and immersive experience.

The insights gathered from previous studies offer valuable strategies for enhancing bicycle simulations in VR. However, there remains ample room for improvement. To enrich the simulation experience, Norén [11] suggests that in future work several sensors could be integrated to simulate more controlling devices than limit it to pedaling rotation and steering such as sensors to measure the braking force or tracking the revolutions per minute (RPM) of the bicycle's rear wheel. Similarly, Hamad [9] emphasizes the importance of realistic steering mechanisms over simplistic button controls to increase the realism of the simulation. Additionally, Hamad proposes incorporating haptic feedback features, such as a controllable fan to simulate lateral wind forces. Furthermore, adjusting the bicycle's front-axis height and the pedaling resistance to simulate uphill riding could be utilized.

To address these areas for improvement, this research aims to develop an even more realistic bicycle simulator by accurately replicating each natural control aspect of real bicycling, including riding speed, steering, rear-wheel resistance adjustment, braking, and leaning, using state-of-the-art sensors and actuators. A further objective is to establish a more precise steering measurement method than the HTC Vive Pro Controller attachment on the handlebars used by Hamad [9]. As this research intends to extend previous work in this field, the simulator is enhanced through the integration of additional sensors and actuators, offering authentic control and haptic feedback to elevate realism.

Additionally, this study focuses on improving ease of use by implementing a "plug-and-play" functionality and enhancing scalability to support the potential integration of new features into the system. Maintainability is also prioritized, simplifying processes like troubleshooting or updating individual components. These goals are pursued through a structured software architecture, which is designed to facilitate seamless initialization, reliable performance, and straightforward utilization in research contexts.

3 Hardware Setup

Creating a realistic simulator necessitated the faithful emulation of physical mechanisms involved in bicycling to engender an immersive simulation experience. Various sensors and actuators were employed to simulate every natural

action of cycling, even in a static bicycle setup. This chapter elucidates and justifies the selection of hardware components pertinent to the project.

3.1 Physical Bicycle

To ensure a realistic riding experience in the VR environment, a physical bicycle frame equipped with various sensors and actuators was utilized (Fig. 1). Additionally, the simulation is displayed on an HTC Vive Pro Eye headset.

The bicycle frame is mounted on a rocker plate, allowing the user to lean to either side. The inclination angle is measured using a BNO055 absolute orientation sensor, which integrates an accelerometer, gyroscope, magnetometer, and a 32-bit microcontroller to provide reliable sensor fusion data [6]. This data is then read and preprocessed by an ESP32 board, a 32-bit microcontroller with integrated Wi-Fi [7]. To simulate braking, a rotary encoder sensor is attached to the brake lever to measure its angle of rotation. This encoder is also interfaced with an ESP32 board. A custom 3D-printed bracket was used to securely mount the rotary encoder, ensuring stability during operation.

Moreover, a 3D-printed custom roll was mounted on the rocker plate. This roll facilitates the realistic simulation of pushing the bicycle back with the right leg within the virtual environment, necessary when the user encounters obstacles such as a building wall in the virtual world. A metal rod embedded within the roll, connected to a magnet, allows determination of the magnetic field's direction using an ams OSRAM AS5600 magnetic sensor. The readings from the AS5600 sensor are also processed by an Inter-Integrated Circuit (I^2C) interface on an ESP32 board.

To determine the moving direction of the roll, a sequence of angle readings is tracked and converted into Cartesian coordinates. Subsequently, the average is calculated to stabilize the readings. The system records whether each angle increases or decreases, storing this trend as bits in a 64-bit integer. By summing these bits, it identifies if the roll is primarily moving forward or backward, using thresholds to prevent rapid state changes.

Furthermore, an Elite Rizer, an indoor gradient simulator, was mounted on the bicycle frame's front axle, allowing reading the bicycle's steering angle. Additionally, the front axle's height of the bicycle can also be adjusted through the Elite Rizer. This feature enables the simulation of uphill gradients in the physical environment corresponding to those in the virtual world.

On the bicycle frame's rear axle, a Direto XR Trainer was installed. This indoor-bike trainer facilitates reading of the current speed and writing of resistance levels to increase pedaling resistance, particularly relevant when simulating uphill terrain in the virtual environment.

Finally, a Wahoo KICKR Bluetooth-Ventilator is employed to provide haptic feedback in the form of an airflow. Adjusting the fan's intensity based on the user's riding speed in the simulation could enhance immersion.

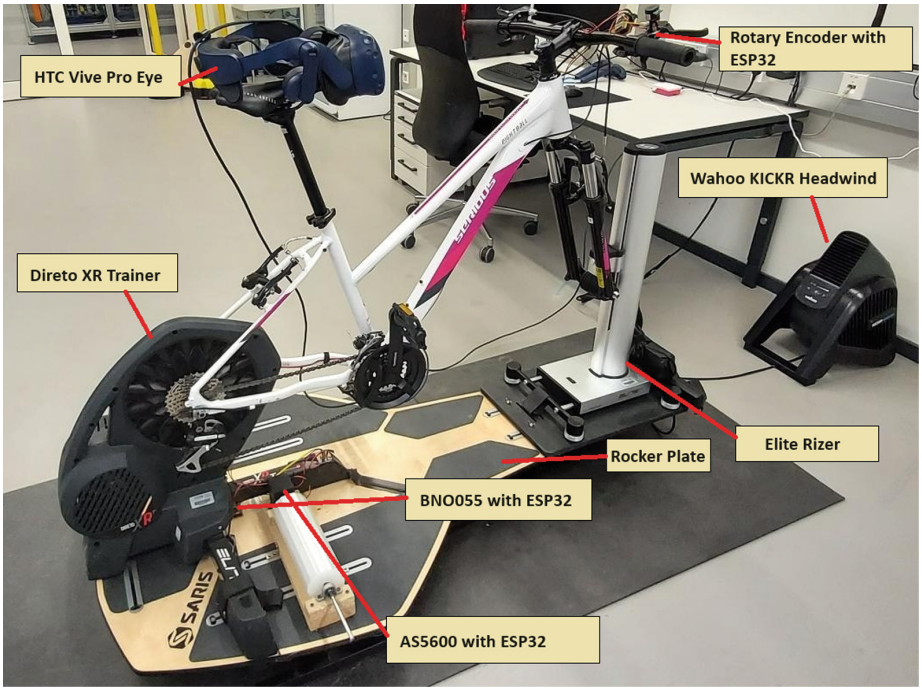

Fig. 1. Hardware setup of the physical bicycle.

3.2 VR Headset

The HTC Vive Pro Eye VR-Headset was chosen for its compatibility with external tracking capabilities provided by the SteamVR Base Station 2.0. This selection was made based on the premise that the user's platform would exhibit mobility during operation. Consequently, camera-based tracking methods, as employed by Meta Quest headsets, were deemed potentially unsuitable for this application. The decision to opt for external tracking ensures enhanced adaptability and precision in monitoring the user's movements within the virtual environment. Moreover, the HTC Vive Pro Eye VR-Headset offers an eye-tracking technology, presenting a potential advantage for further studies within the domains of cycling safety and cycling behavior analysis. Leveraging eye tracking facilitates precise measurement of the visual attention focus exhibited by study participants.

3.3 Raspberry Pi

In order to enhance the modularity, scalability, and maintainability of the simulator, a Raspberry Pi 5 with 8 GB of RAM was used as a distributed system. The Raspberry Pi serves as a centralized hub for collecting data from the simulator's sensors and actuators, preprocessing it for consumption by the Unity simulation. This decentralized approach aims to streamline debugging processes in future iterations. Furthermore, since logging simulation data will be part of the project, it is easy to access the data directly on the raspberry or later after processing with a UDP socket on a target PC.

3.4 Tapo P100

To streamline the initialization process, a TP-Link Tapo P100 smart plug has been integrated as a controllable switch. This setup allows for an automated reset of the entire system, ensuring that each component is freshly calibrated before every operation. The Tapo plug can be seamlessly controlled using the TapoP100 Python library, available on GitHub [8]. In the finalized setup, the smart plug, along with other WI-FI-enabled devices, connects to the Raspberry Pi's WI-FI-access point. This connection enables the smart plug to receive an IP address, which the TapoP100 library can then use to switch the device on or off as needed. Furthermore, after a restart, the Bluetooth Low Energy (BLE) devices will automatically enter pairing mode. This step is crucial for establishing a successful connection.

4 Data Processing

A critical aspect of data processing entailed the retrieval and transmission of data from sensors and actuators. Sensor data is read using C++ code implemented with the Arduino Library and executed on an ESP32 board. However, accessing and manipulating data with actuators posed challenges. The Direto XR Trainer, the Elite Rizer and the Wahoo Kickr Headwind lack comprehensive documentation on data reading and writing protocols. Further elaboration on this process will be discussed in Sect. 4.1.

4.1 Reverse Engineering of the Actuators

In order to comprehend the functionality of Bluetooth Low Energy (BLE) in detail, it was imperative to initiate an exploration of its workings. A comprehensive understanding of BLE is elucidated within the Bluetooth Low Energy Documentation, accessible through the official Bluetooth website [4]. Communication with the BLE devices was facilitated through Python scripts utilizing the Bleak Library. Upon establishing a connection with each actuator, the identification of pertinent Generic Attribute (GATT) profiles became pivotal. A GATT profile delineates a hierarchical structure, organizing data into services and characteristics. Access to a service or its characteristics is facilitated through a Universally Unique Identifier (UUID). Subsequently, the task was to discern the relevant UUIDs of the GATT profiles crucial for controlling the simulator. These were ascertained utilizing the nRF Connect mobile app by Nordic Semiconductor ASA [2] and Bluetooth's Assigned Numbers Documentation [5]. The nRF Connect mobile app and the Assigned Numbers Documentation provide insights into additional information, such as properties of the respective GATT profiles or the data types utilized for input and output.

Characteristics that can be read possess the properties READ or NOTIFY, while those that can be written to possess the property WRITE. Upon reading a characteristic, the return value is delivered as a byte array. A byte array can

contain one or more bytes, yielding diverse values contingent upon the characteristic. The interpretation of byte data varies depending on the manufacturer and specific actuator. Consequently, further processing of these bytes is often requisite to derive meaningful values for subsequent utilization within the simulation. The nRF Connect app already interpreted the bytes within a byte array automatically. However, in instances where no explicit information is provided on how these bytes should be interpreted, the challenge lies in devising a logic to interpret the bytes accurately. The goal is to ensure that the interpreted value closely aligns with or matches the value displayed in the nRF Connect app (Fig. 2).

Furthermore, during the reverse engineering process, it was observed that there might be a slight delay of approximately 2–4 s when querying the speed value of the Direto XR Trainer, which may affect immersion. However, this issue pertains only to reading the speed values from the Direto XR Trainer. The characteristics of the GATT profiles of other actuators, as well as the GATT profile for writing the resistance levels to the Direto XR Trainer, could be read or written without significant latency.

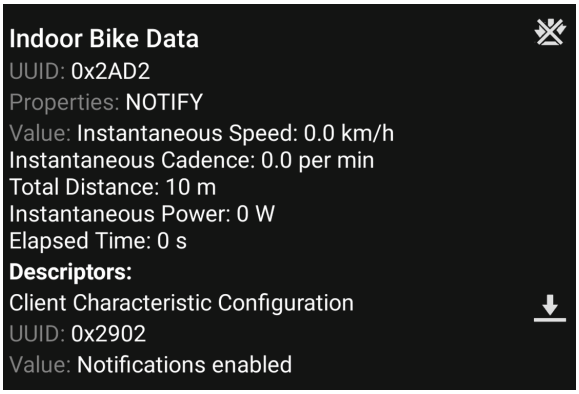

Fig. 2. Byte array values which are displayed in the nRF Connect app.

Another challenge related to the Elite Rizer was determining the correct GATT profile responsible for adjusting the height of the physical bicycle's front axle. The manufacturer of the Elite Rizer used so-called "Hidden UUID's" for some GATT profiles used, for which neither information was available in Bluetooth's Assigned Numbers Documentation nor in the nRF Connect mobile app. Consequently, it was not possible within the scope of this project to determine the correct GATT profile or its associated data type. However, this issue poses a complex problem that will be addressed in future research endeavors.

4.2 Data Transmission Pipeline

A primary objective in developing the bicycle simulator was to process sensor and actuator data in real-time, ensuring minimal latency between data acquisition, preprocessing, and transmission to Unity, or vice versa. Achieving this

necessitated the adoption of a communication protocol capable of transmitting data with minimal delay. Consequently, the User Datagram Protocol (UDP) was selected for this purpose.

UDP is well-suited for this application due to its ability to handle high data rates, which aligns with the high sampling rates of the sensors and actuators. Thus, potential packet loss, which might occur during transmission, is inconspicuous to the user due to the continuous data flow. However, significant latency in data transmission would disrupt the immersive experience.

Once sensor and actuator data are collected on the Raspberry Pi, they are aggregated into a unified JSON object. Subsequently, this JSON object is transmitted as a UDP packet to the Unity simulation (Fig. 3). The frequency of UDP packet transmission may slightly vary, but typically remains within the range of a few milliseconds.

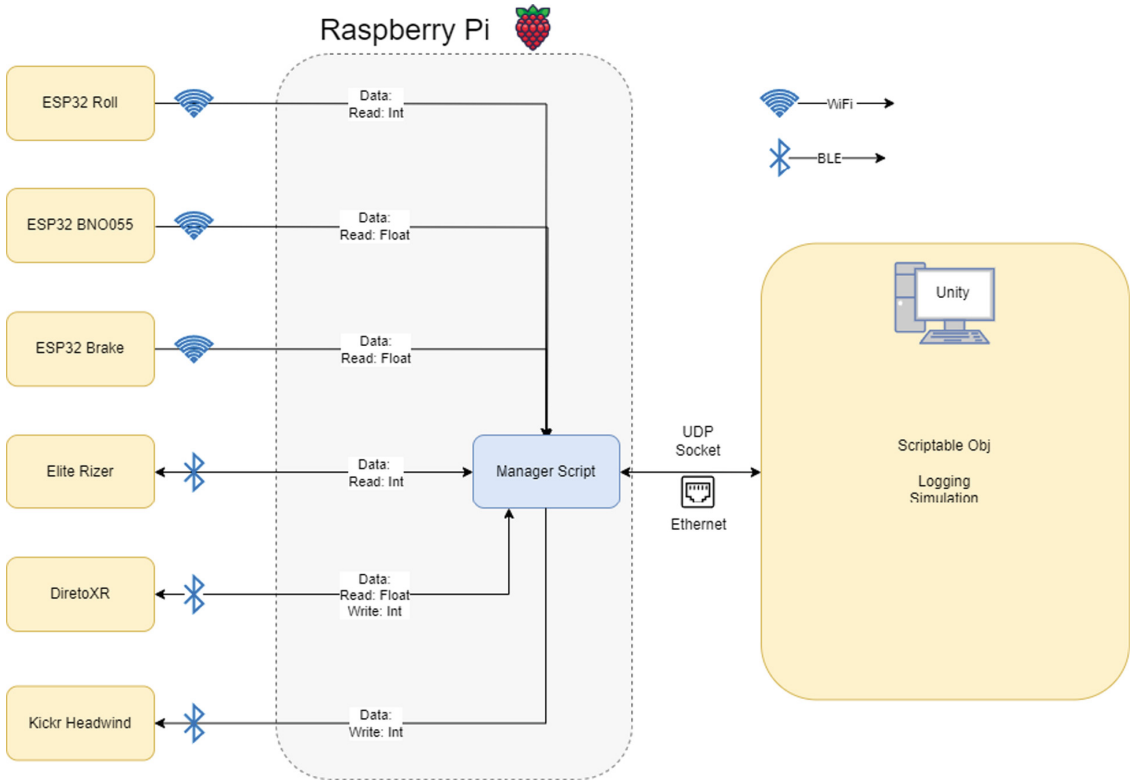

Fig. 3. Visualization of the data flow between the sensors/actuators and the Unity simulation.

5 Implementation in Unity

The unity simulation consists of a 3D-environment in which a bicycle asset is placed. The asset stems from the Simple Bicycle Physics extension from Unity's marketplace [15]. The bicycle prefab provides the necessary interface inputs

which can be used for the collected sensor and actuator data. This data is received from the Raspberry Pi by a UDP socket. To store the data, a scriptable object is utilized, allowing for asynchronous reading and writing of the values within the simulation (Fig. 4). The stored values are continuously overwritten by subsequent values originating from the same data source, which are received within the UDP package. Subsequently, the data is distributed to the relevant points in the virtual bicycle in the simulation. The reverse route is used for data that needs to be sent to the actuators: the Elite Rizer adjusts the bike's height, the Direto XR Trainer controls the pedaling resistance, and the Wahoo KICKR Headwind simulates wind speed depending on the cycling speed. For further details, see Sect. 3.1.

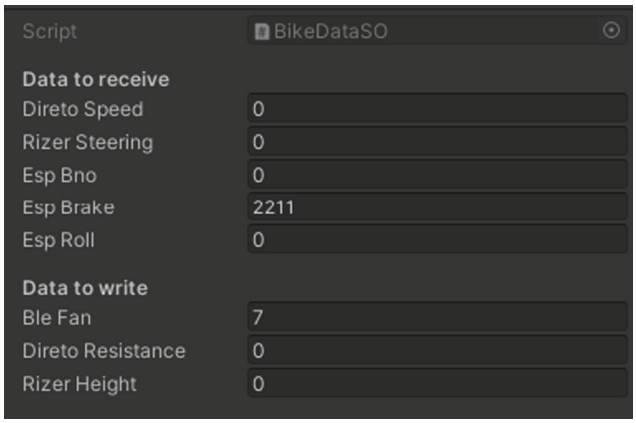

Fig. 4. Scriptable object in unity, displaying incoming and outgoing data.

5.1 Creating Roads with RoadRunner

Mathworks RoadRunner was used to design a virtual environment based on the style of German roads. This software serves as a platform for virtual test-driving, offering a suite of assets and tools to construct 3D road networks. It's noteworthy that most Unity assets typically reflect mostly American or rather stylized road designs, which are not suitable for the bicycle simulator. Lane markings, road signs, and traffic lights must conform to German standards to ensure they are easily recognizable by users. This minimizes distractions for the user caused by mismatched surroundings, and the focus can be on the main purpose of the simulation. RoadRunner offers the essential assets for creating a German traffic environment. Constructing the environment within the 3D editor is straightforward, involving dragging the splines for roads or entering the precise coordinates. The desired road network can then be exported to Unity using a Filmbox (FBX) file. After importing it to Unity, the rest of the environment was prototypically added. The scope of the demo scene in this research was specifically confined to a sample crossroads, featuring German street signs and several buildings. (Fig. 5).

Fig. 5. Sample crossroads with German road signs in an urban environment.

6 Conclusion and Future Work

In summary, the implementation of the VR bicycle simulator has laid the foundation for a realistic and immersive simulation experience by integrating physical controls through sensors and actuators. Consequently, the bicycle simulator fulfills all essential functions of a real bicycle and can be started automatically through plug-and-play functionality. The modular implementation allows the bicycle controller to be utilized in any Unity scenario without deep integration into a Unity project. As this work solely focused on the implementation of the simulator, no user studies regarding the usability of the simulator have been conducted.

Furthermore, as previously mentioned in Sect. 4.1, a latency issue was observed when reading the speed values from the Direto XR Trainer, which could potentially diminish the immersion of the simulation experience for the user. However, this latency is specific to the Direto XR Trainer and does not arise from data transmission via UDP. Nonetheless, the querying of other sensors and actuators operates in real-time with minimal latency.

Building on the current implementation, several avenues for future work have been identified to enhance the VR Bicycle Simulator and explore additional research questions.

Since the latency issues between the Direto XR Trainer and the Raspberry Pi are known, it is crucial for future work to thoroughly investigate these delays to determine whether they are due to hardware limitations or are a result of software inefficiencies during data processing. Furthermore, as mentioned in Sect. 4.1, it was not possible to determine the GATT profile, which is responsible for gradient adjustment of the front axle of the bicycle frame on the Elite Rizer. The next step is to fully integrate the gradient adjustment using BLE. One possible approach for this would be to record and analyze the communication between the Elite Rizer mobile app and the Elite Rizer actuator using the HCI Snoop Log debugging tool, which is particularly available on mobile Android Devices [3]. Once this feature is implemented, it is crucial that the simulator will be tested in a user study to evaluate its usability, realism, and immersion.

If the study results are favorable, further VR scenarios will be developed to conduct empirical studies to evaluate the cycling safety and cycling behavior in certain situations such as urban traffic environments. To create a realistic test environment for these studies, it is essential to implement a traffic simulation for these VR scenarios to simulate interaction with moving cars, buses, and other road users. Additionally, implementing a logging feature for the simulation could significantly simplify the evaluation of the user's cycling behavior. To achieve this, meaningful metrics could be developed for the cycling data, such as cycling speed or braking behavior, captured by the Raspberry Pi and sent to the Unity simulation. Due to the scalability of the software architecture in this project, new data sources that may also be of interest for studies can be seamlessly integrated. Together, these approaches offer a comprehensive collection for future research on many relevant topics.

References

1. Brandt, M.: Innerorts: Jeder 3. Verkehrstote fuhr Rad [Digitales Bild], 14 July 2023. https://de.statista.com/infografik/30408/im-strassenverkehr-innerhalb-von-ortschaften-getoetete-fahrradfahrerinnen-in-deutschland/. Accessed 17 Feb 2024
2. nRF Connect for Mobile (2024). https://play.google.com/store/apps/details?id=no.nordicsemi.android.mcp&hl=de&gl=US. Accessed 16 Feb 2024
3. Android Open Source Project: Verifying and debugging Bluetooth. https://source.android.com/docs/core/connect/bluetooth/verifying_debugging?hl=de. Accessed 14 Aug 2024
4. Bluetooth SIG, Inc.: The Bluetooth LE Primer, January 2023. https://www.bluetooth.com/wp-content/uploads/2022/05/The-Bluetooth-LE-Primer-V1.1.0.pdf, version 1.1.0
5. Bluetooth SIG, Inc.: Bluetooth assigned numbers, February 2024. https://www.bluetooth.com/wp-content/uploads/Files/Specification/HTML/Assigned_Numbers/out/en/Assigned_Numbers.pdf?v=1707997104517. Version Date: 2024-02-14
6. Bosch Sensortec: BNO055 intelligent 9-axis absolute orientation sensor. https://www.bosch-sensortec.com/products/smart-sensor-systems/bno055/. Accessed 14 Aug 2024
7. Espressif Systems: ESP32 overview. https://www.espressif.com/en/products/socs/esp32. Accessed 14 Aug 2024
8. fishbigger: Tapop100 (2024). https://github.com/fishbigger/TapoP100. Accessed 14 Feb 2024
9. Hamad, A.: Two Wheelistic. Ph.D. thesis, University of Michigan (2021). https://deepblue.lib.umich.edu/bitstream/handle/2027.42/167360/Ayah%20Hamad%20-%20Final%20Thesis%20-%20Two%20Wheelistic.pdf?sequence=1
10. Kwon, D.S., et al.: KAIST interactive bicycle simulator. In: Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No. 01CH37164), vol. 3, pp. 2313–2318 (2001). https://doi.org/10.1109/ROBOT.2001.932967
11. Norén, H.: Developing a virtual reality bicycle simulator in unity for traffic safety research integration. Dissertation, Malmö University (2022). https://urn.kb.se/resolve?urn=urn:nbn:se:mau:diva-53350

12. Pearson, L., Gabbe, B., Reeder, S., Beck, B.: Barriers and enablers of bike riding for transport and recreational purposes in Australia. J. Transp. Health **28**, 101538 (2023). https://doi.org/10.1016/j.jth.2022.101538. https://www.sciencedirect.com/science/article/pii/S2214140522002109
13. Pucher, J., Buehler, R.: Cycling towards a more sustainable transport future. Transp. Rev. **37**(6), 689–694 (2017). https://doi.org/10.1080/01441647.2017.1340234
14. Statistisches Bundesamt: Statistischer bericht - verkehrsunfälle zeitreihen - 2013–2022 (2023). https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Verkehrsunfaelle/Publikationen/Downloads-Verkehrsunfaelle/statistischer-bericht-verkehrsunfaelle-zeitreihen-5462403227005.html. Accessed 17 Feb 2024
15. Unity Technologies: Simple bicycle physics (2021). https://assetstore.unity.com/packages/tools/physics/simple-bicycle-physics-206818. Accessed 16 Feb 2024
16. Younes, H., Noland, R.B., Von Hagen, L.A., Sinclair, J.: Cycling during and after COVID: has there been a boom in activity? Transport. Res. F Traffic Psychol. Behav. **99**, 71–82 (2023). https://doi.org/10.1016/j.trf.2023.09.017. https://www.sciencedirect.com/science/article/pii/S1369847823002000